

Universidad de los Andes > FACULTAD DE INGENIERÍA Y CIENCIAS APLICADAS

Arquitectura De Computadores
Proyecto Nº 2
Juego en Assembly sobre Pochoco SoC

Grupo 2

Integrantes:
Edgardo Guianatti
Sebastián Letelier
Florencia Sáez
Cristóbal Toro

Profesores:
Jorge Gomez

20 de Octubre de 2026

Resumen

En este informe se presenta el diseño y la implementación de un juego interactivo de reflejos desarrollado en Assembly, ejecutado por el procesador Espino Core dentro del Pochoco SoC y desplegado en una FPGA Lattice iCE40 HX1K en la tarjeta Nandlangd Go Board. Al comienzo de cada ronda, el juego enciende los cuatro LEDs durante al menos tres segundos, elige un LED objetivo a partir del estado del contador de ciclos, mide el tiempo de respuesta y lo muestra en décimas de segundo en los displays de siete segmentos. Tras diez respuestas correctas se muestra el promedio, si se presiona un botón incorrecto se muestra EE y se reinicia la ronda sin perder el progreso acumulado.

La solución separa el diseño en tres partes: el hardware de integración y temporización (el módulo game_top.v y un contador de ciclos de 32 bits, con reloj nominal de 25 MHz), el programa game.s y el assembler en Python desarrollado por nosotros para convertir game.s en game.hex. El diseño fue sintetizado y programado en la Go Board, probado funcionalmente sobre la placa.

1. Introducción y objetivos

El proyecto requiere integrar el Pochoco SoC con la Go Board y ejecutar un juego interactivo escrito en Assembly para el Espino Core. Para lograrlo, se construyó un assembler propio. El archivo fuente game.s se transforma en game.hex, con una palabra de 32 bits representada en hexadecimal por línea, sin llamar a un assembler RISC-V externo.

1.1 Objetivo General

Diseñar e implementar un juego de reflejos en Assembly, con temporización basada en ciclos de reloj y una herramienta propia de ensamblado, integrándolo en el Pochoco SoC para su ejecución en la FPGA.

1.2 Objetivos Específicos

- Crear un módulo superior game_top.v que instancie pochoco_soc con la imagen sw/game.hex y conecte sus entradas y salidas a los LEDs, botones y displays de la Go Board.
- Implementar una espera inicial de al menos tres segundos usando el contador de ciclos cycle_counter.v, accesible desde el programa en la dirección 0x8000000C.
- Seleccionar el LED objetivo a partir de los dos bits menos significativos del contador de ciclos, evitando una lista fija de LEDs en el programa.
- Medir cada respuesta con el contador de ciclos, desde la escritura del LED objetivo hasta la detección de la pulsación, y expresar el resultado en décimas de segundo.
- Acumular diez respuestas correctas, mostrar su promedio y conservar el progreso cuando ocurra una pulsación incorrecta.
- Desarrollar un assembler propio en Python con codificación de instrucciones, resolución de etiquetas, validación de operandos y pruebas reproducibles.

2. Requisitos Funcionales

La siguiente tabla transforma las exigencias del enunciado en verificaciones concretas. La columna de evidencia resume cómo se comprobó cada requisito y la de estado indica el alcance real de esa comprobación.

Requisito	**Comportamiento esperado**	**Evidencia**	**Estado**
Inicio de ronda	Los cuatro LEDs permanecen encendidos durante al menos 3 s.	Espera por contador de 75.000.000 ciclos (etiqueta wait_three_seconds). Prueba en placa con un botón mantenido al inicio.	Verificado en placa
Selección de objetivo	Después de la espera, solo queda encendido un LED.	Se escribe una máscara 1, 2, 4 u 8 en el registro de LEDs; la selección usa los bits [1:0] del contador.	Implementado, sin registro formal de secuencias.
Medición individual	El contador de inicio se lee inmediatamente después de encender el LED objetivo.	Comparación manual con cronómetro: diferencias aproximadas de 350–450 ms.	Verificado de forma aproximada.
Respuesta correcta	Se detiene la medición, se muestra el tiempo en décimas y se suma una respuesta.	Diez respuestas consecutivas; el tiempo permanece visible mientras se mantiene el botón.	Verificado en placa.
Respuesta incorrecta	Se indica error, se reinicia la ronda y no cambia el número de aciertos previos.	Secuencias de 3 errores + 10 aciertos y de 2 errores + 5 aciertos + 2 errores + 5 aciertos.	Verificado en placa.
Décima respuesta	Se calcula y muestra el promedio de las diez respuestas correctas.	Promedio mostrado en los displays tras el décimo acierto.	Verificado en placa
Assembler propio	Genera game.hex con palabras de 32 bits en hexadecimal, una por línea.	game.s produce 87 palabras y tests/test.s produce 9; formato y errores comprobados (sección 7.4).	Verificado.

3. Arquitectura Propuesta

La arquitectura distingue el flujo de compilación del software y el sistema que se ejecuta físicamente. El archivo game.s se procesa con el assembler que se creó, para producir game.hex. El módulo game_top.v instancia pochoco_soc, que carga esa imagen en su RAM mediante el parámetro MemFile y conecta las señales de la placa.

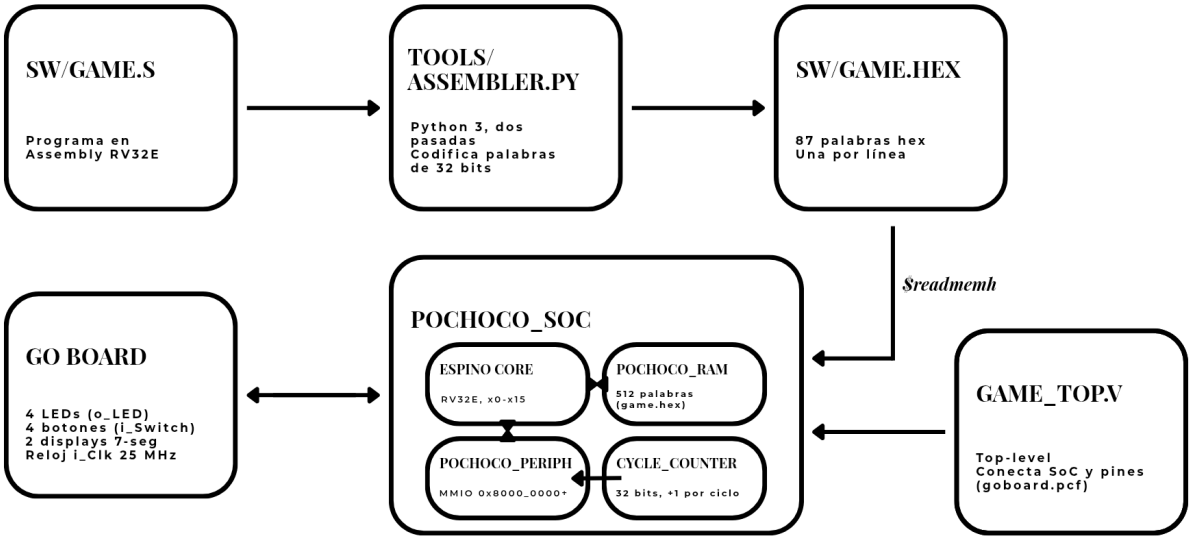


Figura 1. Flujo del software y bloques principales del hardware implementado.

3.1 Módulos y responsabilidades

Componente	Responsabilidad	Detalle en el código
game_top.v	Top-level que instancia pochoco_soc, selecciona la imagen de memoria y conecta las señales de la Go Board.	Parámetro MemFile = "sw/game.hex". Puertos: i_Clk, i_Switch[3:0], o_LED[3:0], los siete segmentos de dos displays y cuatro señales SPI; pines en goboard.pcf.
Pochoco SoC / Espino Core	Ejecutar el programa Assembly y proporcionar acceso a memoria y periféricos.	Core RV32E (x0–x15) y RAM de 512 palabras (NumWords). Un acceso de datos con el bit 31 en 1 es MMIO; con el bit 16 en 0 va a pochoco_periph y con el bit 16 en 1, al esclavo SPI. Reset de encendido con un contador de 4 bits.
Contador de ciclos	Incrementar en cada ciclo de reloj y permitir consultar el conteo.	cycle_counter.v: registro de 32 bits, +1 por flanco de subida de i_Clk, reset asíncrono activo en bajo (rst_ni). Se lee en 0x8000000C; no tiene captura ni reset por software.
Interfaz de E/S	Leer los cuatro botones, controlar los LEDs y mostrar resultados en displays.	pochoco_periph.v: 0x80000000 displays (dos dígitos hexadecimales), 0x80000004 LEDs (4 bits), 0x80000008 botones (lectura de 4 bits) y 0x8000000C contador. Los botones se leen sin anti-rebote ni sincronizador; los displays son activos en bajo y se invierten en pochoco_soc.v.

Componente	Responsabilidad	Detalle en el código
game.s	Implementar el flujo del juego, la selección del objetivo, la temporización y el promedio.	230 líneas y 87 instrucciones; usa los registros x5–x15 (x5 = base MMIO). No usa multiplicación, división ni desplazamientos: la división se hace por restas sucesivas.
Assembler del grupo	Traducir las instrucciones aceptadas a palabras de 32 bits y resolver las etiquetas.	tools/assembler.py (Python 3, 611 líneas): dos pasadas, 31 instrucciones base y 5 pseudoinstrucciones, validación de registros RV32E y de rangos de inmediatos.

3.2 Secuencia de una ronda

Paso	Acción del sistema	Resultado o condición
1. Inicio	Poner el display en 0x00, encender los cuatro LEDs (0xF) y leer el contador como C_espera.	Comienza la espera; el display muestra 00.
2. Espera	Repetir la lectura del contador hasta que C − C_espera ≥ 75.000.000 (comparación sin signo).	Los LEDs permanecen encendidos al menos 3 s.
3. Liberación	Esperar a que ningún botón esté presionado.	Evita contar un botón mantenido de la ronda anterior.
4. Objetivo	Leer el contador, conservar sus dos bits bajos y convertirlos en la máscara 1, 2, 4 u 8.	Queda definido el LED y botón esperados.
5. Inicio de medida	Escribir la máscara en el registro de LEDs y leer el contador inmediatamente después.	Se guarda C_inicio; solo queda encendido el LED objetivo.
6. Espera de botón	Leer los botones hasta obtener un valor distinto de cero y leer el contador (C_fin).	Es correcta si la máscara leída es idéntica a la del objetivo.
7.a. Correcta	Calcular ΔC, convertirlo a décimas por restas sucesivas, sumarlo a la suma acumulada, mostrarlo (saturado en 99) e incrementar los aciertos.	Se espera que se suelte el botón y comienza otra ronda.
7.b. Incorrecta	Apagar los LEDs, mostrar EE y esperar que se suelte el botón.	Se vuelve al inicio de ronda sin modificar aciertos ni suma.
8. Fin del juego	Con diez aciertos, apagar los LEDs, calcular el promedio (suma ÷ 10 por restas) y mostrarlo.	El programa queda en un bucle infinito; se debe reiniciar la placa.

4. Contador de ciclos y temporización

El programa no depende de bucles de espera calibrados por instrucciones, la referencia temporal es el contador de ciclos de 32 bits del hardware (cycle_counter.v), que se lee por MMIO en la dirección 0x8000000C y se incrementa en cada flanco de subida de i_Clk.

4.1 Frecuencia y espera de tres segundos

Sea f_{clk} la frecuencia del reloj que impulsa al SoC. En el RTL, el reloj del sistema es directamente la entrada de la placa (assign clk = i_Clk en pochoco_soc.v, pin 15 en goboard.pcf), sin PLL ni divisores, y la implementación usa el valor nominal f_{clk} = 25 MHz del oscilador de la Go Board. Como el contador aumenta una vez por ciclo, el número de ciclos de una espera de tres segundos es:

$$N_{3s} = 3 \cdot f_{clk} = 3 \times 25\,000\,000 = 75\,000\,000$$

Al iniciar cada ronda se encienden los cuatro LEDs, se lee el contador (x8) y se repite la lectura (x12) hasta que la resta x12 − x8 sea mayor o igual que 75.000.000. El umbral se calculó a partir de la frecuencia nominal y está definido en game.s como .equ THREE_SECONDS, 75000000.

El contador tiene 32 bits, comienza en cero cuando termina el reset de encendido (reset asíncrono activo en bajo) y se lee con una instrucción lw a la dirección 0x8000000C; la lectura se registra en pochoco_periph.v. Como no existe un reset por software, el programa trabaja siempre con diferencias entre dos lecturas.

A 25 MHz el contador completa su período en 2^{32} / 25.000.000 ≈ 171,8 s. Como las diferencias se calculan con sub sobre 32 bits (aritmética módulo 2^{32}), el resultado es correcto aunque el contador pase por cero, siempre que el intervalo medido sea menor que ese período.

Magnitud	**Valor**	**Uso en game.s**
1 segundo	25.000.000 ciclos	—
3 segundos	75.000.000 ciclos	THREE_SECONDS
1 décima de segundo	2.500.000 ciclos	DECISECOND
Período del contador	2^{32} ciclos ≈ 171,8 s	Desbordamiento (sin efecto en las restas)

4.2 Instante de inicio y tiempo de respuesta

El LED objetivo y la captura de C_inicio son dos instrucciones consecutivas del programa. Primero se escribe la máscara en el registro de LEDs y a continuación se lee el contador. No existe un registro de hardware que realice ambas acciones en un único evento, la diferencia entre ambas operaciones es de una instrucción (unos pocos ciclos de reloj), despreciable frente a los 2.500.000 ciclos de una décima de segundo.

Cuando se detecta una pulsación (lazo wait_button, que lee el registro de botones hasta obtener un valor distinto de cero), se lee de inmediato el contador C_fin y se calcula la diferencia con aritmética de 32 bits:

$$\Delta C \;=\; (C_{fin} \;-\; C_{inicio})\; mod\; 2^{w},\;\; w \;=\; 32$$

Aquí w representa el ancho del contador en bits. Con w = 32 la resta se realiza directamente con la instrucción sub y el resultado se interpreta sin signo, de modo que el desbordamiento del contador es transparente mientras el intervalo medido sea menor que 2^{32} ciclos (≈ 171,8 s). La lectura de los botones y la del contador son instrucciones consecutivas, por lo que C_fin corresponde al instante en que se detectó la pulsación.

4.3 Conversión a décimas de segundo

El programa muestra la cantidad entera de décimas de segundo, truncando hacia abajo. Como una décima equivale a f_{clk} / 10 = 2.500.000 ciclos, la conversión es:

$$T_{i} \;=\; \left\lfloor \frac{\Delta C_{i}}{2\,500\,000} \right\rfloor \;=\; \left\lfloor \frac{10 \cdot \Delta C_{i}}{f_{clk}} \right\rfloor$$

Por ejemplo, una lectura de 15 significa 15 décimas, es decir, 1,5 segundos. Este formato evita depender de que los displays permitan representar un punto decimal. Como el juego no usa división, la conversión se hace por restas sucesivas (etiqueta convert_time): mientras ΔC ≥ 2.500.000 se resta esa constante y se incrementa un contador, que al terminar contiene T_i. Para mostrarlo se limita a 99 solo en la visualización (el valor acumulado no se limita) y se convierte a dos dígitos decimales empaquetados, se resta 10 repetidamente sumando 16 al registro de decenas, de modo que 15 décimas se escribe como 0x15 y se ve “15” en los displays. No se escribe el valor binario 15 (0x0F), porque cada display recibe un dígito hexadecimal de cuatro bits.

4.4 Promedio de diez respuestas correctas

Para el promedio, el programa acumula en x7 los tiempos individuales T_i ya convertidos a décimas y lo calcula una sola vez, al décimo acierto, dividiendo por 10 mediante restas sucesivas (etiqueta average_loop). El promedio en décimas es:

$$T_{prom} = \left\lfloor \frac{\sum_{i=1}^{10} T_i}{10} \right\rfloor$$

La suma incluye solo rondas correctas: una pulsación errónea no modifica x7 ni el contador de aciertos x6, de modo que la ronda fallida no afecta el promedio. Como cada T_i se trunca antes de sumarse, el promedio puede quedar hasta una décima por debajo del que se obtendría con los ciclos exactos, es una simplificación que evita sumar y dividir cantidades de ciclos de gran magnitud. No hay riesgo de desbordamiento: la suma de diez valores T_i es muy inferior a 2^{32}, y el valor mostrado se satura en 99.

Ejemplo, si ΔC = 41.000.000 ciclos, entonces T = ⌊41.000.000 / 2.500.000⌋ = 16 décimas (1,6 s) y el display recibe 0x16. Si las diez respuestas correctas fueran 12, 15, 9, 20, 14, 11, 16, 13, 18 y 12 décimas, la suma sería 140 y el promedio ⌊140 / 10⌋ = 14 décimas, que se escribe como 0x14.

5. Selección del LED objetivo

El enunciado permite obtener una semilla desde el contador de ciclos. El LED objetivo se elige directamente a partir de los dos bits menos significativos del contador de ciclos, leído cuando termina la espera de tres segundos y todos los botones están liberados. Como ese instante depende de cuándo el jugador soltó el botón, la secuencia de objetivos no es una lista fija codificada en el programa.

- Lectura: lw x8, 12(x5) obtiene el contador y andi x8, x8, 3 conserva solo sus dos bits menos significativos, con un valor entre 0 y 3.
- Conversión a máscara: una cadena de comparaciones transforma el índice 0, 1, 2 o 3 en la máscara one-hot 0x1, 0x2, 0x4 u 0x8, que coincide con la asignación de bits de LEDs y botones (bit 0 = LED/botón 1 … bit 3 = LED/botón 4).
- Uso de la máscara: la misma máscara se escribe en el registro de LEDs y luego se compara con la lectura de botones, la respuesta es correcta sólo si ambas son idénticas.

6. Diseño del programa Assembly

El programa game.s está escrito para RV32E y se organiza como un único flujo con etiquetas que delimitan cada fase del juego. Las constantes se definen con .equ (direcciones MMIO, THREE_SECONDS y DECISECOND) y el registro x5 guarda la base MMIO, de modo que los periféricos se acceden con los desplazamientos 0, 4, 8 y 12.

Etiqueta	**Responsabilidad**
_start	Cargar la base MMIO y poner en cero el contador de aciertos (x6) y la suma de décimas (x7).
new_round	Poner el display en 0x00, encender los cuatro LEDs y leer el contador como inicio de la espera.
wait_three_seconds	Esperar 75.000.000 ciclos mediante la resta sin signo del contador.
wait_release	Esperar a que no haya botones presionados.
(selección) y selected	Elegir el LED con los bits bajos del contador, escribir su máscara en los LEDs y leer el contador como C_inicio.
wait_button	Leer los botones hasta detectar una pulsación y leer el contador (C_fin), si los botones no coinciden con la máscara, saltar a wrong_button.
convert_time	Convertir ΔC a décimas por restas sucesivas de 2.500.000.
time_converted … bcd_done	Acumular T en x7, saturar a 99 para mostrar, convertir a dos dígitos decimales (BCD) y escribir en el display; incrementar x6.
release_correct	Esperar que se suelte el botón y volver a new_round (no se ejecuta al décimo acierto).
wrong_button, release_wrong	Apagar los LEDs, mostrar EE (0xEE = 238) y esperar que se suelte el botón; vuelve a new_round sin modificar x6 ni x7.
calculate_average … average_bcd_done	Dividir x7 entre 10 por restas sucesivas, saturar a 99, convertir a BCD y mostrar el promedio.
finished	Bucle infinito (j finished); el juego solo se reinicia reiniciando la placa.

El uso de registros del programa se resume en la siguiente tabla:

Registro	**Uso**
x5	Base MMIO (0x80000000).
x6	Contador de respuestas correctas.
x7	Suma de los tiempos de las respuestas correctas, en décimas.
x8	Auxiliar: C_espera, índice del LED (0–3) y C_inicio.
x9	Auxiliar: umbral de 3 s, C_fin y luego ΔC.

Registro	Uso
x10	Máscara del LED y botón esperados (1, 2, 4 u 8).
x11	Botones leídos.
x12–x15	Auxiliares: contador actual, constantes, décimas (x13) y conversión a dos dígitos decimales (x14, x15).

6.1 Pseudocódigo del programa

```
inicio:
  base_mmio <- 0x80000000; aciertos <- 0; suma_ds <- 0
nueva_ronda:
  display <- 0x00; leds <- 0xF
  C_espera <- leer_contador()
  esperar_hasta(leer_contador() - C_espera >= 75.000.000)   # sin signo
  esperar_hasta(botones == 0)
  indice <- leer_contador() AND 3
  objetivo <- mascara(indice)            # 1, 2, 4 u 8
  leds <- objetivo
  C_inicio <- leer_contador()
  esperar_hasta(botones != 0)
  C_fin <- leer_contador()
  si botones != objetivo:
    leds <- 0; display <- 0xEE
    esperar_hasta(botones == 0)
    ir a nueva_ronda                     # conserva aciertos y suma_ds
  delta <- C_fin - C_inicio; t <- 0
  mientras delta >= 2.500.000: delta <- delta - 2.500.000; t <- t + 1
  suma_ds <- suma_ds + t
  display <- BCD(min(t, 99))
  aciertos <- aciertos + 1
  si aciertos == 10: ir a promedio
  esperar_hasta(botones == 0)
  ir a nueva_ronda
promedio:
  leds <- 0
  prom <- 0; mientras suma_ds >= 10: suma_ds <- suma_ds - 10; prom <- prom + 1
  display <- BCD(min(prom, 99))
  repetir para siempre                   # finished
```

6.2 Tratamiento de errores y continuidad

Una pulsación incorrecta se identifica comparando la máscara de botones leída con la máscara del LED objetivo. Como la comparación es de igualdad exacta, presionar dos botones a la vez también se considera error. En ese caso se apagan los LEDs, los displays muestran EE y el programa espera que se suelte el botón antes de volver a new_round, por lo que el siguiente intento vuelve a tener su fase inicial de tres segundos con los cuatro LEDs y una nueva selección de objetivo. Los registros x6 (aciertos) y x7 (suma de décimas) no se modifican, de modo que la ronda fallida no se incluye en el promedio.

Tras una respuesta correcta, el tiempo individual se muestra en décimas de segundo y permanece visible mientras el botón sigue presionado, el número de respuestas correctas

aumenta en uno y, al soltar el botón, comienza la siguiente ronda, con el display en 00. Una vez alcanzadas diez respuestas correctas se apagan los LEDs, se calcula y muestra el promedio y no se inicia una ronda adicional: el programa queda en un bucle infinito hasta reiniciar la placa.

7. Diseño del assembler propio

El assembler recibe game.s y produce game.hex. El enunciado exige una palabra de 32 bits en hexadecimal por línea y prohíbe invocar un assembler RISC-V externo. Está escrito en Python 3 usando solo la biblioteca estándar (argparse, re, sys y pathlib) y se ejecuta con el comando python3 tools/assembler.py sw/game.s sw/game.hex, al terminar con éxito informa la cantidad de instrucciones generadas.

7.1 Flujo del assembler

- Leer el archivo fuente y eliminar comentarios (# y //) y líneas vacías.
- Primera pasada: registrar las constantes .equ y las etiquetas con su dirección (cada instrucción ocupa 4 bytes y li ocupa una o dos palabras según su valor), validar la sintaxis de cada línea y comprobar que el programa no supere las 512 palabras de la RAM.
- Segunda pasada: codificar cada instrucción según su formato (R, I, S, B, U o J) con los opcodes y campos funct3/funct7 de RV32I, expandiendo las pseudoinstrucciones li, mv, j, nop y ret.
- Resolver etiquetas e inmediatos: el desplazamiento de un salto es la dirección de la etiqueta menos la dirección de la instrucción, se comprueban los rangos y la alineación a múltiplos de 2.
- Escribir cada palabra como ocho dígitos hexadecimales en minúsculas, una por línea, en game.hex.
- Ante una instrucción desconocida, un operando inválido, un valor fuera de rango o una etiqueta duplicada o indefinida, detener la generación y terminar con código de salida 1 sin escribir el archivo de salida.

7.2 Sintaxis soportada

Elemento	**Sintaxis implementada**	**Validaciones**
Comentarios	# o // hasta el final de la línea.	Se ignoran las líneas vacías.
Etiquetas	nombre: al inicio de la línea (letras, dígitos y guion bajo), puede ir sola o antes de una instrucción.	Una etiqueta o constante duplicada produce error.
Constantes y directivas	.equ NOMBRE, valor. También se aceptan .text y .globl _start.	Cualquier otra directiva produce error.

Elemento	Sintaxis implementada	Validaciones
Registros	x0–x15 y alias ABI (zero, ra, sp, gp, tp, t0–t2, s0/fp, s1, a0–a5).	x16 o superior produce error (RV32E).
Instrucciones	add, sub, slt, sltu, xor, or, and; addi, slti, sltiu, xori, ori, andi; lw, lh, lhu, lb, lbu, sw, sh, sb; beq, bne, blt, bge, bltu, bgeu; jal, jalr, lui, auipc; y las pseudoinstrucciones li, mv, j, nop y ret.	Se verifica el número de operandos de cada instrucción. No se codifican desplazamientos ni la extensión M, que el juego no necesita.
Inmediatos	Decimal, hexadecimal (0x), binario (0b), negativos y nombres definidos con .equ; operandos de memoria como desplazamiento(registro).	12 bits con signo en tipos I y S; 0 a 0xFFFFF en lui y auipc; li hasta 32 bits.
Saltos/branches	Etiqueta o desplazamiento numérico; el desplazamiento se calcula relativo al PC de la instrucción.	13 bits en branches y 21 bits en jal; alineación a 2 bytes.
Salida	Una palabra de 32 bits por línea, ocho dígitos hexadecimales en minúsculas y sin prefijo.	Compatible con $readmemh de pochoco_ram.v; máximo 512 palabras.

7.3 Codificación

La codificación sigue el formato base RV32I que ejecuta el Espino Core (espino_decoder.v reconoce los opcodes LOAD 0x03, OP-IMM 0x13, AUIPC 0x17, STORE 0x23, OP 0x33, LUI 0x37, BRANCH 0x63, JALR 0x67 y JAL 0x6F). La siguiente tabla muestra instrucciones reales de game.s, tal como las traduce el assembler, junto con su palabra en game.hex.

Dirección	Línea Assembly	Campos de la instrucción	Palabra en game.hex
0x00000000	li x5, DISPLAY (1/2) → lui x5, 0x80000	opcode 0x37, rd = 5, imm[31:12] = 0x80000	800002b7
0x00000004	li x5, DISPLAY (2/2) → addi x5, x5, 0	opcode 0x13, funct3 = 000, rd = 5, rs1 = 5, imm = 0	00028293
0x00000018	sw x8, 4(x5)	opcode 0x23, funct3 = 010, rs1 = 5, rs2 = 8, imm = 4	0082a223
0x00000028	lw x12, 12(x5)	opcode 0x03, funct3 = 010, rd = 12, rs1 = 5, imm = 12	00c2a603
0x00000030	bltu x13, x9, wait_three_seconds	opcode 0x63, funct3 = 110, rs1 = 13, rs2 = 9, desplazamiento = −8	fe96ece3

7.4 Pruebas del assembler

Prueba	Entrada/caso	Resultado	Estado
Programa de prueba	tests/test.s (9 palabras: li, sw, addi, bne, j).	La salida regenerada es idéntica a tests/test.hex.	Verificado
Programa completo	sw/game.s.	87 palabras; idéntico a sw/game.hex.	Verificado
Etiquetas	Saltos hacia atrás (bltu wait_three_seconds, bne loop) y hacia adelante (beq … selected).	Desplazamientos resueltos; el juego funciona en la placa con la imagen generada.	Verificado (de forma indirecta)
Inmediatos	addi x1, x0, −1; andi x2, x1, 0xFF; addi x1, x0, 5000.	Acepta negativos y hexadecimales (fff00093 y 0ff0f113); rechaza 5000 por estar fuera de rango de 12 bits.	Verificado
Errores de sintaxis	Mnemónico desconocido, operandos de menos, registro x16, etiqueta indefinida, etiqueta duplicada y directiva .word.	Mensaje con número de línea y código de salida 1; no se genera la salida. (Para la etiqueta indefinida, el mensaje es «Valor inválido».)	Verificado
Formato de salida	sw/game.hex.	87 líneas, cada una con ocho dígitos hexadecimales.	Verificado
Límite de tamaño	Programa de 513 instrucciones nop.	Error: programa demasiado grande (513 palabras; máximo 512).	Verificado
Integración	Regenerar game.hex y reconstruir el bitstream.	La imagen generada se ejecuta en la Go Board (sección 9).	Verificado en placa

8. Integración con la FPGA y modificaciones al RTL

El punto de partida indicado por el enunciado es el repositorio del Pochoco SoC. Como grupo, agregamos un top-level propio, game_top.v, que instancia pochoco_soc con MemFile = "sw/game.hex" y expone los puertos que necesita la Go Board (reloj, cuatro botones, cuatro LEDs, dos displays de siete segmentos y el puerto SPI), con los pines definidos en goboard.pcf.

```
// Extracto de rtl/game_top.v (se omiten los puertos de segmentos y SPI)
module game_top (
    input  wire       i_Clk,
    input  wire [3:0] i_Switch,
    output wire [3:0] o_LED,
    // o_Segment1_A..G, o_Segment2_A..G, i_SPI_SCLK, i_SPI_MOSI, i_SPI_CS_n, o_SPI_MISO
);
    pochoco_soc #(.MemFile("sw/game.hex")) u_pochoco (
        .i_Clk(i_Clk), .i_Switch(i_Switch), .o_LED(o_LED),
        // ... resto de los puertos, conectados uno a uno
    );
endmodule
```

8.1 Hardware agregado y conexiones

- Reloj: el SoC usa i_Clk (pin 15) directamente como reloj del sistema, sin PLL ni divisor, la frecuencia nominal es de 25 MHz. Un contador de 4 bits genera el reset de encendido (rst_ni).
- Acceso a periféricos: el Espino Core accede a los periféricos con cargas y almacenamientos; una dirección con el bit 31 en 1 es MMIO y pochoco_periph.v decodifica el desplazamiento (0x0 display, 0x4 LEDs, 0x8 botones, 0xC contador).
- Contador de ciclos: el módulo cycle_counter.v, de 32 bits y reset asíncrono, está instanciado dentro de pochoco_periph.v y su valor se devuelve al leer 0x8000000C.
- Botones, LEDs y displays: los botones i_Switch[3:0] se leen directamente sin anti-rebote ni sincronizador, los LEDs usan los cuatro bits bajos escritos, el display recibe 8 bits (dos dígitos hexadecimales) que pochoco_periph.v convierte a siete segmentos con la función hex2seg y pochoco_soc.v invierte porque los displays de la placa son activos en bajo.
- Síntesis y programación: el Makefile usa Yosys con goboard.pcf, icepack e iceprog. Al modificar game.s se debe regenerar game.hex y ejecutar make clean antes de reconstruir, porque el Makefile no declara game.hex como dependencia de la síntesis.

8.2 Mapa de periféricos (MMIO)

Dirección	Periférico	Operación	Detalle
0x80000000	Displays de siete segmentos	Escritura	8 bits: [7:4] display 1 y [3:0] display 2, un dígito hexadecimal cada uno.
0x80000004	LEDs	Escritura	4 bits: bit 0 = LED 1 … bit 3 = LED 4.
0x80000008	Botones	Lectura	4 bits: bit 0 = botón 1 … bit 3 = botón 4.
0x8000000C	Contador de ciclos	Lectura	32 bits, +1 por ciclo de reloj.

8.3 Recursos y procedimiento de construcción

La síntesis y el place & route finalizaron correctamente. La utilización reportada de la FPGA se muestra a continuación.

Recurso	Utilización
ICESTORM_LC	1175 / 1280 (91 %)
ICESTORM_RAM	12 / 16 (75 %)
SB_IO	27 / 72 (37 %)
SB_GB	3 / 8 (37 %)

El procedimiento de construcción y programación, desde la raíz del repositorio, es el siguiente:

```
python3 tools/assembler.py sw/game.s sw/game.hex
make clean
make game_top.asc
make game_top.bin
make prog
```

8.4 Organización del repositorio

Ruta	Contenido
rtl/game_top.v	Top-level del juego: instancia pochoco_soc con sw/game.hex.
rtl/cycle_counter.v	Contador de ciclos de 32 bits.
rtl/pochoco_soc.v, pochoco_ram.v, pochoco_periph.v, pochoco_spi_slave.v	SoC, RAM de 512 palabras, periféricos MMIO y esclavo SPI.
rtl/espino_core/	Espino Core (RV32E): etapas IF e ID, ALU, banco de registros, unidad de carga/almacenamiento, controlador y decodificador.
sw/game.s, sw/game.hex	Código fuente Assembly del juego (230 líneas) e imagen generada (87 palabras); el directorio incluye además programas de ejemplo (blink, 7seg, buttons_leds).
tools/assembler.py	Assembler propio en Python.

Ruta	Contenido
tests/test.s, tests/test.hex	Programa de prueba del assembler y su salida.
goboard.pcf, Makefile	Pines de la Go Board y flujo Yosys, nextpnr, icepack e iceprog.

9. Plan de pruebas y demostración

El plan de pruebas se organizó según los puntos del enunciado. Las pruebas del juego se ejecutaron funcionalmente sobre la Go Board y las del assembler con la herramienta en línea de comandos, el repositorio no incluye testbenches RTL.

Prueba	**Procedimiento**	**Resultado esperado**	**Resultado observado**
Fase inicial	Reiniciar el SoC y mantener un botón presionado durante el inicio.	Los cuatro LEDs permanecen encendidos ≥ 3 s.	El juego espera el tiempo inicial y la liberación del botón. La duración exacta no se midió con instrumentos.
Ronda correcta	Esperar al LED objetivo, presionar su botón y mantenerlo.	La ronda termina y el tiempo se muestra en décimas.	El tiempo permanece visible mientras el botón está presionado y no se registra otro acierto.
Diez aciertos	Completar diez respuestas correctas consecutivas.	Se muestra el promedio calculado con esas diez mediciones.	Se muestra el promedio final.
Botón incorrecto	Tres errores iniciales y luego diez aciertos; dos errores, cinco aciertos, dos errores y cinco aciertos.	Se indica error y se reinicia la ronda sin perder aciertos anteriores.	Los errores no se cuentan, se conserva el progreso y se muestra el promedio.
Botón mantenido entre rondas	Mantener un botón presionado entre una ronda y la siguiente.	No se selecciona el siguiente LED hasta soltarlo.	La selección espera la liberación del botón.
Medición de tiempo	Comparar con un cronómetro manual.	Valores coherentes con el tiempo real.	Diferencias aproximadas de 350–450 ms, compatibles con el inicio manual del cronómetro; no es una calibración.
Dos botones simultáneos	Comparación de máscaras en el código.	Se considera una respuesta incorrecta.	Comprobado por inspección del código; sin prueba física controlada.
Pseudoaleatoriedad	Ejecutar varias partidas y registrar los objetivos.	No se observa una secuencia fija idéntica en todas las partidas.	La selección usa los bits bajos del contador; no se registraron secuencias de forma sistemática (sección 5).
Assembler	Ejecutar las pruebas de sintaxis y codificación (sección 7.4).	Salida correcta para instrucciones válidas y rechazo de entradas inválidas.	Verificado con tests/test.s, game.s y casos de error.
Regeneración de imagen	Regenerar game.hex desde game.s.	El resultado coincide con la imagen entregada; los cambios se reflejan al reconstruir.	La salida es idéntica a sw/game.hex; para reconstruir el bitstream se usa make clean.

9.1 Alcance y limitaciones de la verificación

- Las mediciones se contrastaron con un cronómetro, las diferencias de 350–450 ms incluyen el tiempo de reacción de quien acciona el cronómetro, por lo que no son una calibración de la frecuencia del reloj.
- La frecuencia de 25 MHz es el valor nominal del oscilador de la placa
- No se registraron secuencias de LEDs ni se hizo una prueba física con dos botones simultáneos.
- El repositorio no incluye testbenches de simulación, la verificación del hardware es funcional, sobre la placa.
- La resolución de visualización es de una décima y los displays solo muestran valores entre 00 y 99, los tiempos mayores se saturan en 99 únicamente para mostrarlos.

10. Conclusiones

Se implementó un juego de reflejos funcional en Assembly RISC-V sobre el Espino Core y el Pochoco SoC, utilizando los periféricos físicos de la Go Board. El proyecto integra software de bajo nivel, acceso a periféricos mediante MMIO, un contador de ciclos de 32 bits incorporado al hardware y un assembler propio en Python que genera la imagen de memoria de 87 palabras. Las pruebas realizadas sobre la placa confirmaron el funcionamiento de las rondas, la recuperación frente a errores, el manejo de botones mantenidos y la presentación del promedio final.

Entre las limitaciones, la selección del LED depende del estado del contador y no de un generador pseudoaleatorio independiente, los displays solo muestran valores entre 00 y 99, la división se realiza por restas sucesivas, la partida termina tras diez aciertos y exige reiniciar la placa, y la medición depende de la frecuencia efectiva del reloj.